



NestJS Hotel SaaS Backend

This document serves **two purposes**: 1. **Runbook** – how to run, operate, and troubleshoot the system
2. **Project Documentation** – a chronological and architectural record of what has been built so far

The backend is considered **stable and production-ready (MVP)**.

Project Overview

Purpose

A **backend-only SaaS platform** for hotel management.

- Tenant = **Hotel**
- Users = **Hotel staff**
- Architecture is **single-tenant today, multi-tenant by design**

Key Principles

- Strong isolation by tenant
- Secure authentication & authorization
- DTO-first, validation-driven API
- Clean modular NestJS architecture

Technology Stack (Final)

Layer	Technology
Runtime	Node.js v24
Framework	NestJS v10
ORM	TypeORM v0.3.x
Database	PostgreSQL
Auth	JWT (<code>@nestjs/jwt</code> , <code>passport-jwt</code>)
Security	bcrypt, Helmet, ValidationPipe
OS	Linux (WSL)

High-Level Architecture

Client

↓

```

REST API (NestJS Controllers)
  ↓
Guards (JWT)
  ↓
Services (Business Logic)
  ↓
TypeORM Repositories
  ↓
PostgreSQL

```

Cross-cutting layers: * DTO validation * Serialization interceptor * Global pipes & middleware

Module Breakdown (Chronological)

Stage 1 – Tenant Foundation

Tenant Entity

Fields: * `id` (uuid) * `code` (unique, human-readable identifier) * `name` * `isActive` * `createdAt`, `updatedAt`

Why `code`?

- Used in JWT payload
- Safer than exposing UUIDs
- Enables future subdomain or header-based tenancy

Result

- CRUD endpoints tested
 - Tenant lifecycle fully functional
-

Stage 2 – User Management

User Entity

Fields: * `id` * `fullName` * `email` (unique) * `password` (bcrypt hashed) * `role` (ADMIN / STAFF) * `isActive` * `tenant` (ManyToOne → Tenant) * `createdAt`, `updatedAt`

Security Decisions

- Password stored **hashed only**
- Password **never returned** via API

Implementation: * `@Exclude()` on entity field * Global `ClassSerializerInterceptor`

Endpoints

- `POST /users` - create staff/admin
 - `GET /users` - JWT protected
-

Stage 3 – Authentication (MVP Complete)

Login Flow

1. User submits email + password
2. User fetched by email
3. Password compared using bcrypt
4. JWT issued on success

JWT Payload

```
{
  sub: userId,
  email,
  role,
  tenantCode
}
```

JWT Strategy

- `passport-jwt`
- Secret loaded via `ConfigService.getOrThrow`
- Token validated per request

Guards

- `JwtAuthGuard` protects routes
- Auth module fully isolated

Result

✅ Login works ✅ Token validation works ✅ Tenant context preserved

Global Security Configuration

Enabled Globally

- `ValidationPipe`
- Whitelisting
- DTO enforcement
- `ClassSerializerInterceptor`
- Helmet
- Compression

Explicit Decisions

- No `select: false` on password
 - Serialization used instead
 - Prevents accidental leakage
-

Current API Surface (Stable)

Auth

- `POST /auth/login`

Tenants

- `POST /tenants`
- `GET /tenants`
- `GET /tenants/:id`
- `PATCH /tenants/:id`
- `DELETE /tenants/:id`

Users

- `POST /users`
 - `GET /users` (JWT required)
-

Operational Runbook

Environment Variables

```
DATABASE_HOST=  
DATABASE_PORT=  
DATABASE_USER=  
DATABASE_PASSWORD=  
DATABASE_NAME=  
JWT_SECRET=  
JWT_EXPIRES_IN=
```

Running the Application

```
npm install  
npm run start:dev
```

Production:

```
npm run build
npm run start:prod
```

Database Operations

- TypeORM auto-sync used (MVP)
- Future: migrations only

To reset database (DEV only):

```
DROP SCHEMA public CASCADE;
CREATE SCHEMA public;
```

Common Issues & Fixes

Password Appearing in Response

✓ Ensure: * `@Exclude()` present * `ClassSerializerInterceptor` registered * Interceptor added **before** `app.listen()`

JWT Always Invalid

✓ Check: * `JWT_SECRET` matches * Token not expired * Strategy registered in AuthModule

Multi-Tenancy Readiness

Already implemented: * Tenant entity * Tenant-bound users * Tenant code in JWT

Ready for: * Schema-per-tenant * DB-per-tenant * Request-scoped tenant resolution

What Is Intentionally Deferred

- Refresh tokens
- Logout / revocation
- RBAC guards
- Rate limiting
- Audit logs

These were deferred to **protect core correctness**.

Next Logical Stages

1. Role-based guards (ADMIN vs STAFF)
2. Refresh token rotation
3. Hotel domain modules
4. Rooms
5. Reservations
6. Guests
7. Billing

Current Status

Backend Authentication & Tenant Core: COMPLETE (MVP)

System is safe to extend.

End of Document